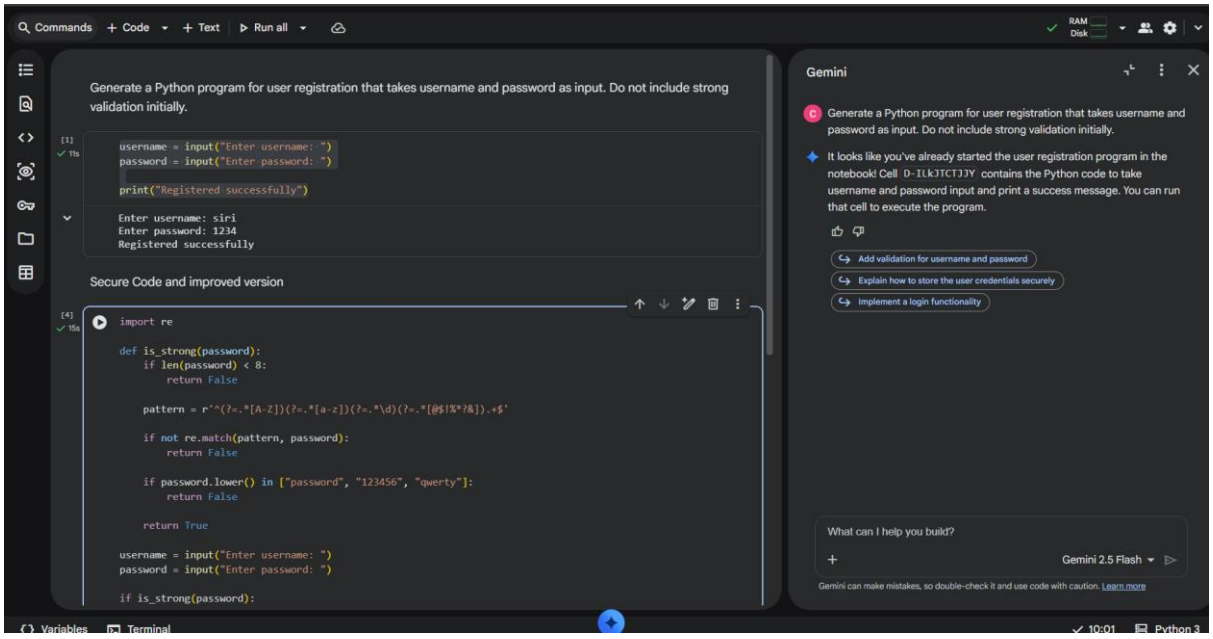
Assignment -20.3

Name : K.Naga Chaitanya
Hall Ticket No.: 2303A51292
Batch No.: 05

Lab 20 – Security Testing: Identifying Vulnerabilities in AI- Generated Code
Task 1 – Password Strength & Validation Check
Task:
Analyze an AI-generated user registration script for weak password handling.
Instructions:
• Prompt AI to generate a Python user registration program
• Check whether:
o Password length is validated
o Weak passwords are allowed
• Suggest secure improvements such as:
o Minimum length checks
o Use of regular expressions
o Rejecting common passwords
Expected Output:
A secure registration script with strong password validation rules.

Prompt:
Generate a Python program for user registration that takes username and password as input. Do not include strong validation initially.
AI Generated Code:



```
        return False

    if password.lower() in ["password", "123456", "qwerty"]:
        return False

    return True

username = input("Enter username: ")
password = input("Enter password: ")

if is_strong(password):
    print("Strong password accepted")
else:
    print("Weak password rejected")
```

```
... Enter username: Chaithu
    Enter password: Chaithu#1349
    Strong password accepted
```

```
Enter username: Chaithu
Enter password: 123445

Weak password rejected
```

Explanation

- No password rules → weak passwords allowed
- Example: 123, abc, password all accepted

Task 2 – Insecure File Upload Handling

Task:

Evaluate an AI-generated file upload program for security risks.

Instructions:

• Prompt AI to generate a Python or Flask-based file upload script

• Identify vulnerabilities such as:

o No file type validation

o Unrestricted file size

• Implement secure measures:

o File extension validation

o Size limits

o Safe file storage paths

Expected Output:

A secure file upload implementation that prevents malicious uploads.

Prompt:

Generate a Python file upload validation script without security checks.

AI Generated Code:

Generate a Python file upload validation script without security checks.

```
filename = input("Enter file name: ")
print("File uploaded:", filename)
```

```
Enter file name: 12345
File uploaded: 12345
```

```
allowed_extensions = ["png", "jpg", "pdf"]

filename = input("Enter file name: ")

ext = filename.split(".")[-1]

if ext.lower() in allowed_extensions:
    print("File accepted")
else:
    print("Invalid file type")
```

```
Enter file name: 123456
Invalid file type
```

Generate a Python file upload validation script without security checks.

```
filename = input("Enter file name: ")
print("File uploaded:", filename)
```

```
Enter file name: 12345
File uploaded: 12345
```

```
allowed_extensions = ["png", "jpg", "pdf"]

filename = input("Enter file name: ")

ext = filename.split(".")[-1]

if ext.lower() in allowed_extensions:
    print("File accepted")
else:
    print("Invalid file type")
```

```
Enter file name: 12345.png
File accepted
```

Explanation

- Any file allowed
- Dangerous files like .exe accepted

Task 3 – Real-Time Application: API Security Review

Scenario:

An AI-generated API endpoint is used in a web application.

Instructions:

• Review the API code for:

o Missing authentication

o Insecure HTTP methods

o Lack of rate limiting

• Prompt AI to suggest security improvements such as:

o Token-based authentication
o Input validation
o Rate limiting

Expected Output:

A security-reviewed API endpoint with documented vulnerabilities and fixes.

Prompt:

Generate a Python API-like function that returns sensitive data without authentication.

AI Generated Code:

Generate a Python API-like function that returns sensitive data without authentication.

```
[8]
def get_data():
    return "Sensitive Data"

print(get_data())
```

```
Sensitive Data
```

```
[10]
VALID_TOKEN = "12345"

def get_data(token):
    if token != VALID_TOKEN:
        return "Unauthorized"
    return "Secure Data"

token = input("Enter token: ")
print(get_data(token))
```

```
Enter token: 123
Unauthorized
```

Generate a Python API-like function that returns sensitive data without authentication.

```
[8]
def get_data():
    return "Sensitive Data"

print(get_data())
```

```
Sensitive Data
```

```
[11]
VALID_TOKEN = "12345"

def get_data(token):
    if token != VALID_TOKEN:
        return "Unauthorized"
    return "Secure Data"

token = input("Enter token: ")
print(get_data(token))
```

```
Enter token: 12345
Secure Data
```

Explanation

- Anyone can access data
- No authentication

Task 4 – Cross-Site Scripting (XSS) Check

Task:

Evaluate an AI-generated HTML form with JavaScript for XSS vulnerabilities.

Instructions:

• Ask AI to generate a feedback form with JavaScript-based output.
• Test whether untrusted inputs are directly rendered without escaping.
• Implement secure measures (e.g., escaping HTML entities, using CSP).

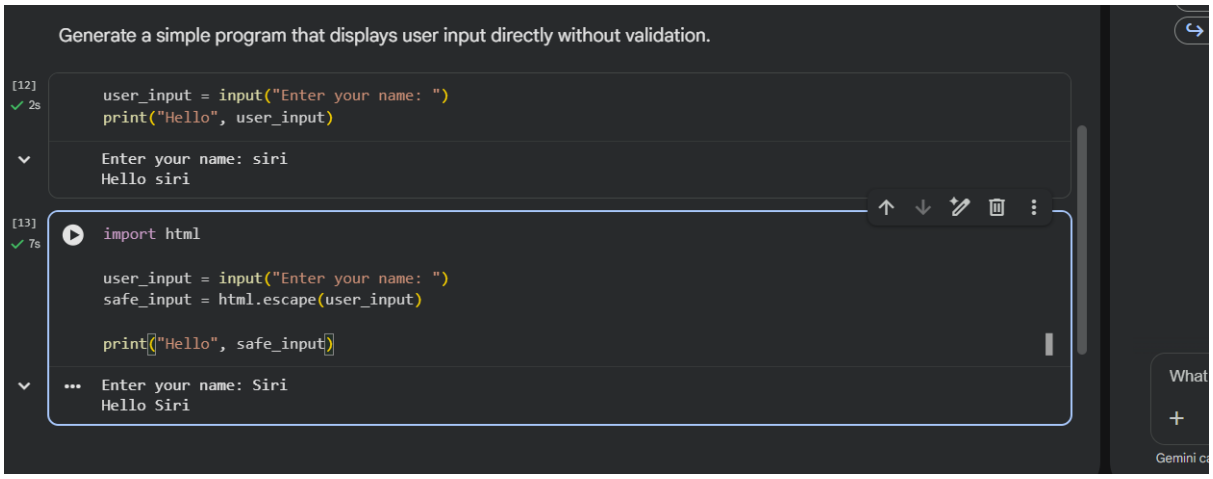
Expected Output:

• A secure form that prevents XSS attacks.

Prompt:

Generate a simple program that displays user input directly without validation.

AI Generated Code:



Explanation:

- In web apps → <script> can run

Task 5 – SQL Injection Prevention

Task:

Test an AI-generated script that performs SQL queries on a database.

Instructions:

• Ask AI to generate a Python script using SQLite/MySQL to fetch user details.
• Identify if the code is vulnerable to SQL injection (e.g., using string concatenation in queries).
• Refactor using parameterized queries (prepared statements).

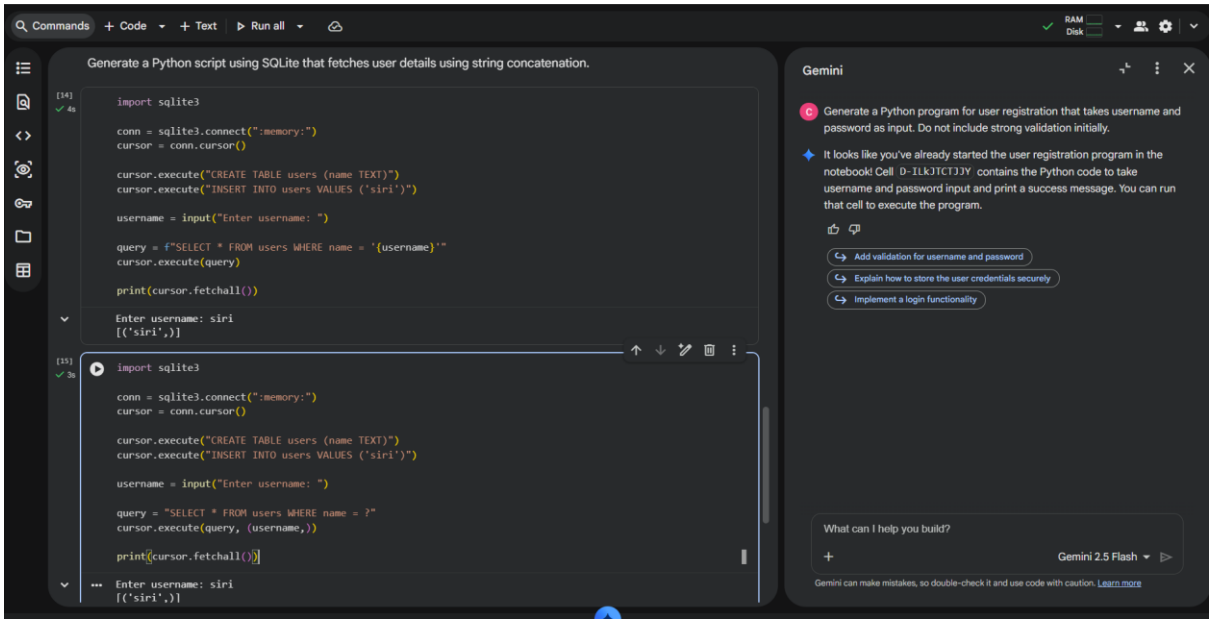
Expected Output:

• A secure database query script resistant to SQL injection.

Prompt:

Generate a Python script using SQLite that fetches user details using string concatenation.

AI Generated Code:



Explanation

- Input like ' OR '1'='1 bypasses security